ENSF-381: Assignment 3

Winter 2026

Department of Electrical and Software Engineering

Schulich School of Engineering

Due: March 27, 2026, at 11:59 PM

Objective

In this assignment, you will build the Sweet Scoop Ice Cream Shop application using React. The LMS will allow students to explore courses, enroll in them, and view testimonials. You will create reusable components, manage state, and implement dynamic interactions while adhering to React best practices.

Administrative Details

- The submission should be a ***single compressed file (.zip)*** that includes the src folder and resources (e.g., images), ready for execution without modifications.
- Name the submitted file as Assignment4_ENSF381_Section#_Group# Example: Assignment4_ENSF381_L02_Group01.zip
- Submit ONLY ONE copy.
- Include group members' names and UCIDs as comments in index.js.
- *Do not modify component names* or add extra styling/functionality beyond requirements.

1. Project Setup (5 Marks)

A. Create React App

```
npx create-react-app sweet-scoop-app
cd sweet-scoop-app
```

B. File Structure

1. Create components and data directories inside src.
2. Add flavors.js and reviews.js to src/data. (See the data files below)
3. Create public/images and add add ice cream images (e.g., vanilla.jpg, chocolate.jpg.).

C. Install Dependencies

```
npm install react-router-dom
```

- List all dependencies you install, including react-router-dom, in readme.txt with installation commands.
- Where a detail is not explicitly specified, you are expected to infer a reasonable implementation and include whatever is needed to satisfy the requirements, such as the necessary state variables, props, event handlers, and component structure.

Data Files

1. **flavors.js**

```
const flavors = [
  {
    id: 1,
    name: "Vanilla Dream",
    price: "$4.99",
    description: " Classic creamy vanilla ice cream made with fresh ingredients.",
    duration: "8 weeks",
    image: "images/vanilla.jpg"
  },
  // Add 9 more flavors...
];

export default flavors;
```

2. **reviews.js**

```
const reviews = [
  {
    customerName: "Alice Johnson",
    flavorName: " Vanilla Dream",
    review: "The ice cream was delicious and very creamy!",
    rating: 5
  },
  // Add 3 more reviews...
];

export default reviews;
```

2. Homepage Requirements (30 Marks)

Components:

A. Header Component

a. Display the Logo and Shop name inside <header> element.

b. Create a navigation section **below the header** using a with class name: className="navbar".

c. The navigation bar should contain links (**Home, Flavors, Login**) using **React Router**.

B. MainSection Component

a. About Sweet Scoop: Brief description of the system.

b. Featured Flavors: Display 3 random ice cream flavors from flavors.js each time the page loads. Wrap the featured flavors inside a container with class name: className="flavor-grid".

c. Customer Reviews: Display 2 random reviews from reviews.js each time the page loads (use useEffect). Include the customer name, review, and star rating (e.g., ★★★★☆).

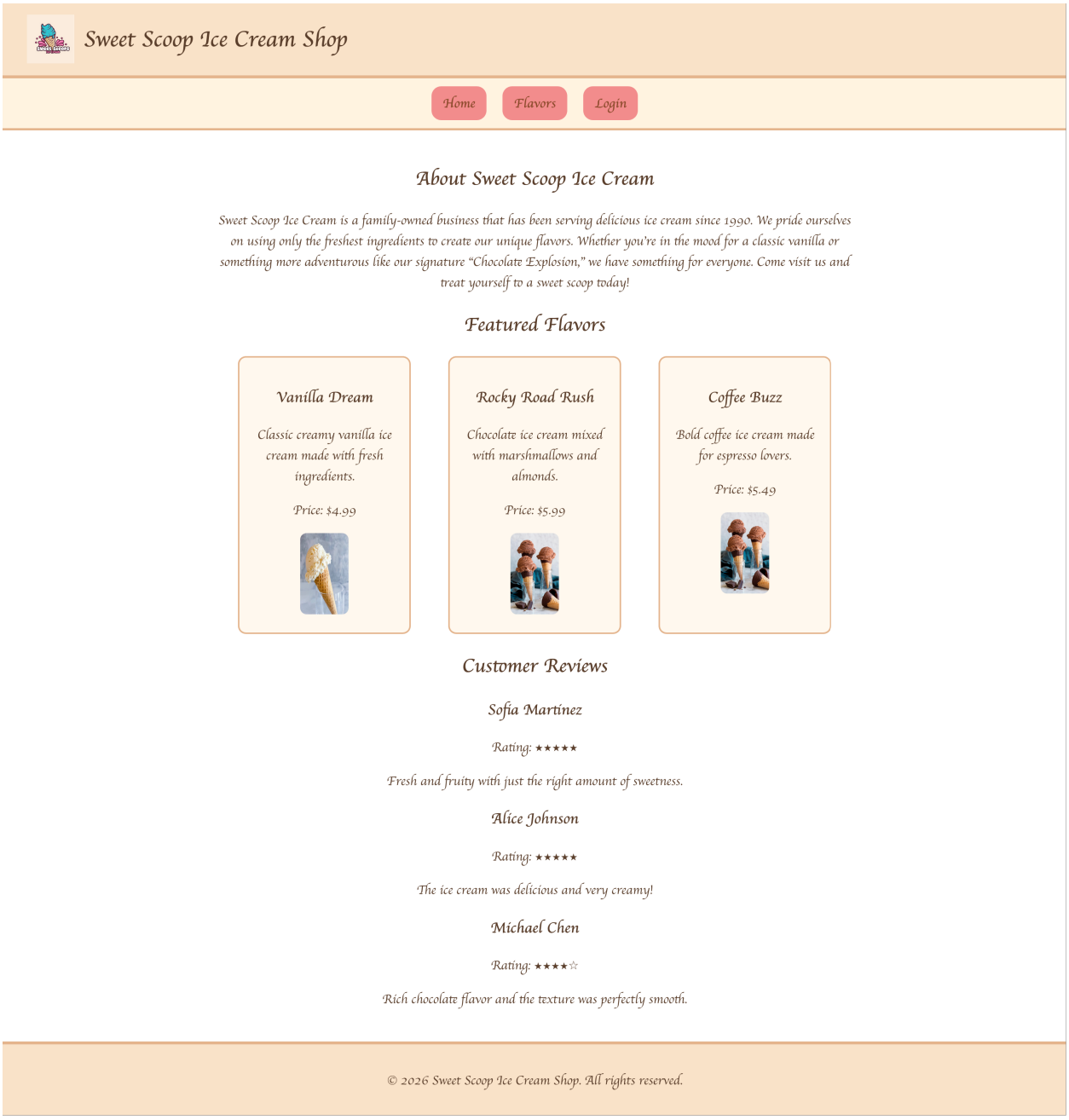
d. Wrap the MainSection content inside a container with class name: className="main-section"

C. Footer Component

a. Copyright notice at the bottom.

Structure of Homepage.js

```
<div>
  <Header />
  <MainSection />
  <Footer />
</div>
```



3. Flavors Page Requirements (40 Marks)

Components

A. FlavorItem Component

a. Displays ice cream image, flavor name, price, and "Add to Order" button.

b. Each flavor should be displayed inside a container with class name: `className="flavor-card"`

c. Hover Effect: Show flavor description on onMouseEnter (use useState) and hide it again on onMouseLeave.

B. OrderItem Component

a. Represents an item in the Order List.

b. Displays flavor name, quantity, and price.

c. Include a “Remove Item” button. <button className="remove">Remove Item</button>

d. Clicking “Remove Item” decreases quantity; remove item if quantity becomes 0.

C. OrderList Component

a. Lists all items added to the order using OrderItem.

b. Calculates and displays total order price (sum of all ordered flavors).

c. Uses useEffect to save/load order data from local storage.

d. Wrap the OrderList content inside a container with class name: className="order-list"

D. FlavorCatalog Component

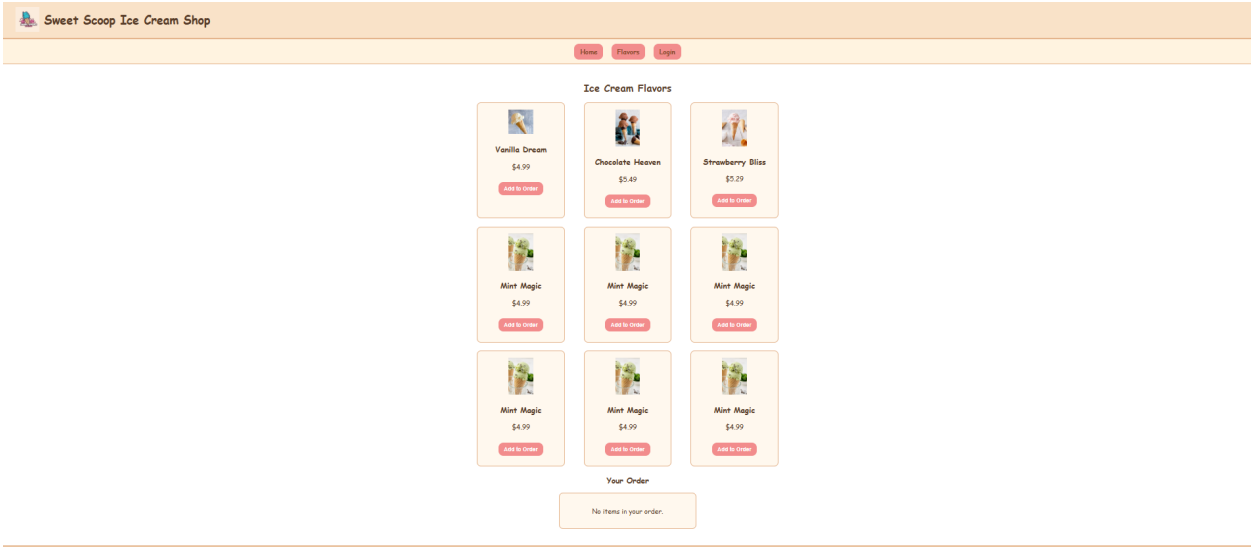
a. Renders all flavors from flavors.js using FlavorItem.

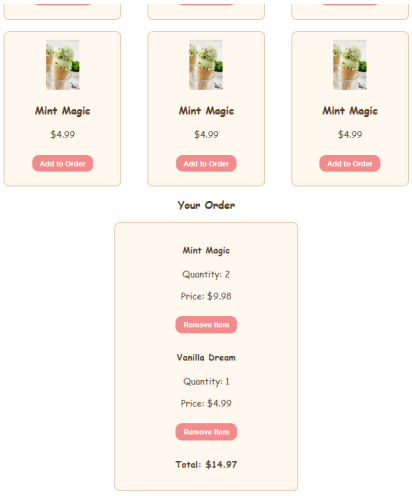
b. Wrap all flavor cards inside a container with class name: className="flavor-grid"

This container should display 3 **flavor cards per row**.

Structure of FlavorsPage.js

```
<div className="flavors-page">
  <Header />
  <div className="content">
    <FlavorCatalog />
    <OrderList />
  </div>
  <Footer />
</div>
```





onMouseEnter:

4. Login Page Requirements (20 Marks)

Components

A. LoginForm Component

a. UI Elements:
   - Username input field.
   - Password input field.
   - Login button.
   - Forgot Password link (non-functional).

b. Validate inputs:
   - Username and password cannot be empty.
   - Password must be at least 8 characters.

c. API Integration:
   - Fetch user data from https://jsonplaceholder.typicode.com/users.
   - Validate credentials against the API's username and email fields. Username should match the API username. Password should match the API email.

d. Implementation Details
   - Use useState for form state management.
   - Use useEffect to handle API calls and redirects.
   - Style the form to match the **Sweet Scoop theme** (reuse CSS classes from Homepage).

B. DisplayStatus Component

a. Accept the following props:
   - type: "success" or "error".
   - message: Text to display.

b. Render the login message inside a styled . If the type is "success", display the message in green; if it's "error", display it in red.

c. Display the appropriate success or error message based on the type prop.

C. Login Feedback and Redirect Behavior

a. Manage the username, password, message, and message type locally in LoginForm using useState.

b. Validate inputs and perform the API request inside LoginForm.

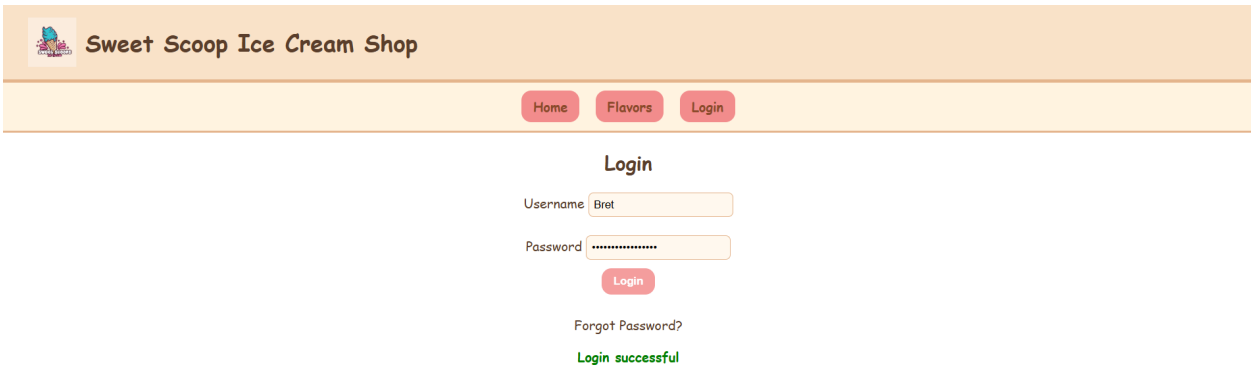
c. Determine the appropriate type and message values in LoginForm, and pass them to DisplayStatus.

d. Conditionally render DisplayStatus only when there is a message to show.

e. Use useEffect to handle API calls and redirect to /flavors after 2 seconds on successful login.

Structure of LoginPage.js

```
<div>
  <Header />
  <LoginForm />
  <Footer />
</div>
```



CSS Styling Requirements (10 Marks)

A. CSS File Setup

- Create a CSS file named style.css inside the src folder: src/style.css.
- Import the CSS file in index.js to apply styles globally: import "./style.css";
- Copy the provided Sweet Scoop styles into style.css as shown below.

```
body {
  margin: 0;
  font-family: 'Baloo 2', cursive;
  /* background: linear-gradient(#87ceeb, #eaf6ff); */
  color: #5a3e2b;
}

/* Header with logo and title */
header {
  background: #f9e2c8;
  display: flex;
  align-items: center;
  gap: 10px;
  padding: 15px 30px;
  border-bottom: 4px solid #e4b48c;
  font-weight: bold;
}
```

```
header h1 {
  margin: 0;
  font-size: 28px;
  font-weight: bold;
}

/* Navigation bar under header */
.navbar {
  background: #fff3e0;
  padding: 10px;
  text-align: center;
  border-bottom: 3px solid #e4b48c;
}

.navbar a {
  text-decoration: none;
  color: #8b4a2b;
  font-weight: bold;
  margin: 0 10px;
}

/* Form styling */
form {
  text-align: center;
  margin-bottom:220px;
}

input {
  background-color: #fff8ee;
  border: 1px solid #e4b48c;
  padding: 6px;
  border-radius: 6px;
}

/* Buttons and links */
a,
button {
  display: inline-block;
  margin-top: 10px;
  margin-bottom: 10px;
  padding: 8px 14px;
  border-radius: 12px;
  border: none;
  background: #f28c8c;
  color: white;
```

```
  font-weight: bold;
  text-decoration: none;
  cursor: pointer;
}

button.remove {
  background: #c94c4c;
}

button:hover,
a:hover {
  opacity: 0.85;
}

/* Footer */
footer {
  background: #f9e2c8;
  text-align: center;
  padding: 15px;
  border-top: 4px solid #e4b48c;
}

img{
  max-width:60px;
}

.main-section {
  max-width: 800px;
  margin: 40px auto;
  text-align: center;
}

.content{
  max-width: 800px;
  margin: 40px auto;
  text-align: center;
}

.flavor-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
  justify-items: center;
  margin: 20px;
}
```

```
.flavor-card {
  background: #fff8ee;
  border: 2px solid #e4b48c;
  border-radius: 10px;
  padding: 15px;
  width: 180px;
  text-align: center;
  margin: 0 auto;
}

.order-list{
  background: #fff8ee;
  border: 2px solid #e4b48c;
  border-radius: 10px;
  padding: 15px;
  width: 300px;
  margin: 0 auto;
  text-align: center;
}
```